计算机串口使用与测量

【实验属于电子测试平台与工具】

湖南大学信息科学与工程学院

计科 2102 甘晴void （学号 202108010206）

0.环境搭建

在实验开始之前，安装好Ubuntu 20.04操作系统。（这个没有难度）

但要提醒的是，这个ubuntu是xubuntu，而且虚拟硬盘只有10GB的大小，

如果想沿用到操作系统OS的Lab实验中要注意了，可能会出现存储不够的现象。

（1）安装VirtualBox虚拟化软件以及该虚拟化软件的扩展包

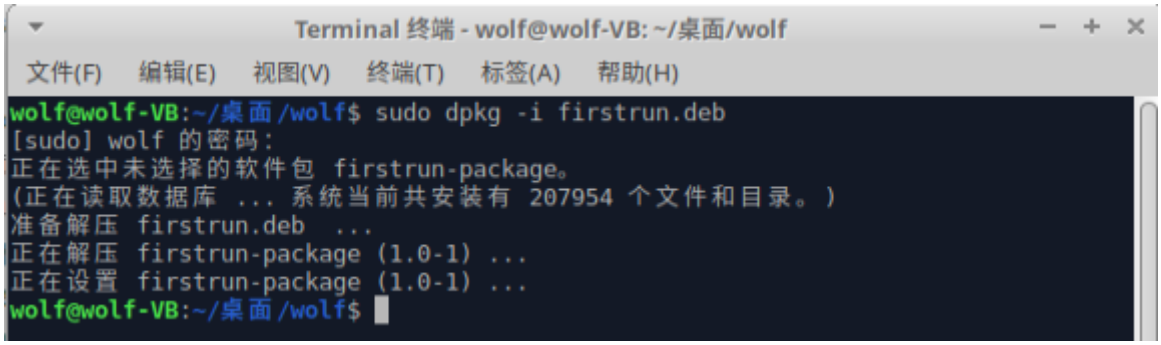
（2）安装Linux(Xubuntu发行版)虚拟机

请安装ubuntu-20.04.4，就是串口部分第一次课上老师给的优盘里拷贝的那个版本的ubuntu，各位同学请互相拷贝安装。

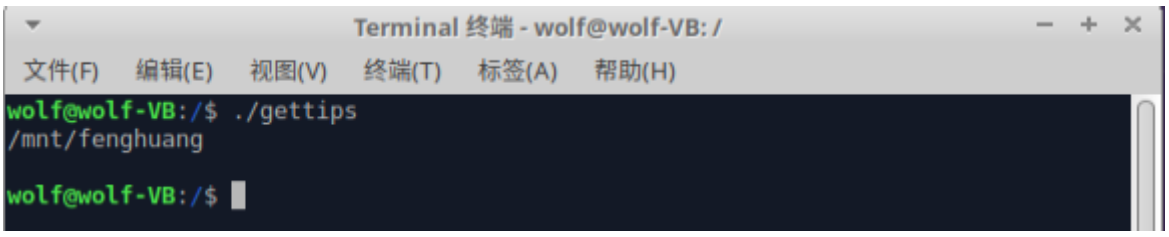
1.linux操作系统平台

（1）安装firstrun.deb包

使用指令sudo dpkg -i firstrun.deb 进行安装



（2）运行根目录下的/gettips

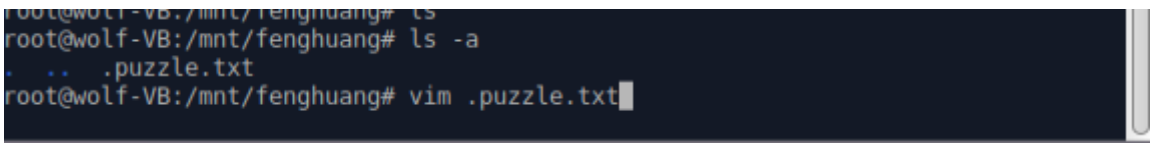


（3）切换到root权限进入目录

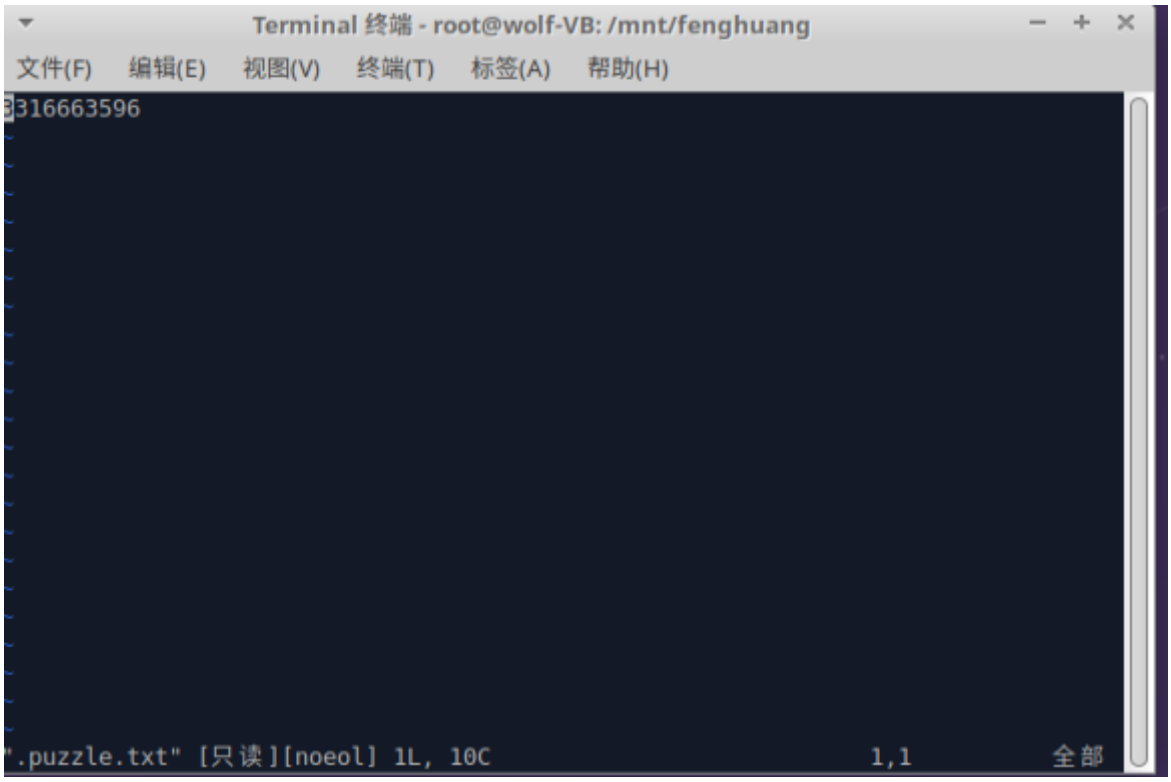
运行指令切换到2中提供的目录，切换到root用户（直接进入权限不够），进入该目录

（4）使用ls打开文件

ls查看并使用vim打开文件.puzzle.txt



发现结果如下

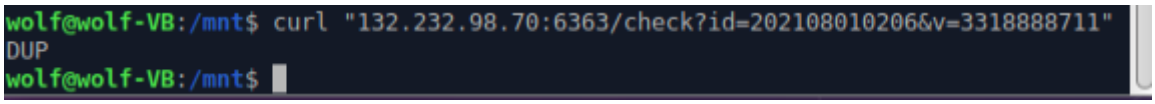


（5）使用curl提交结果

使用如下命令提交

> *curl "132.232.98.70:6363/check?id=202108010206&v=3318888711"*

提交后返回OK，重复提交返回DUP



转换为16进制为C5D23907

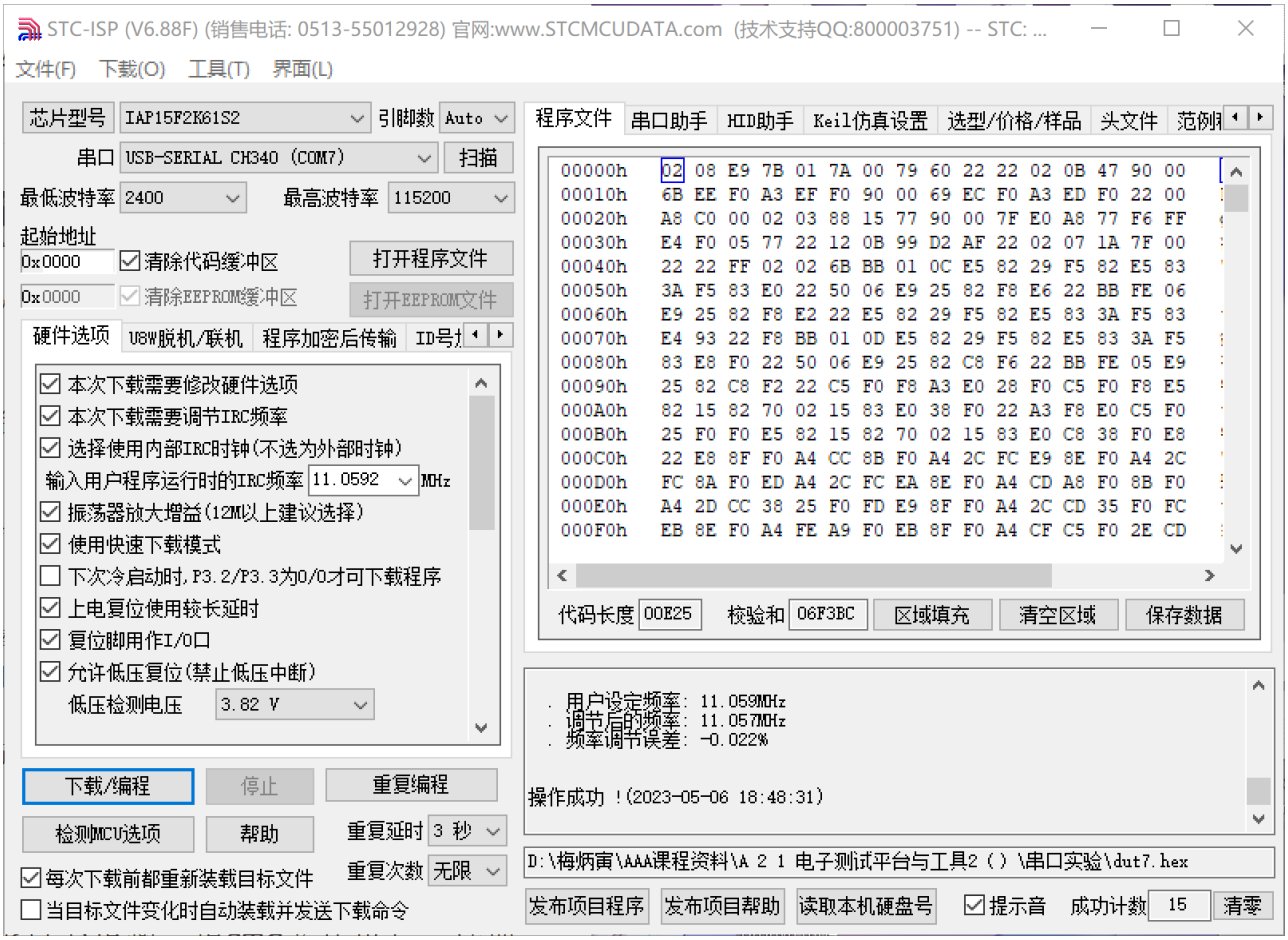
2.Linux平台串口数据接收

（1）下载程序

按照指导书步骤，向STC单片计算机板下载程序。

【注意：按下载按键之后，一定要在板子上按“restart”按键，否则无法下载】

如下所示，显示“操作成功”表示已经成功向板子发送程序。

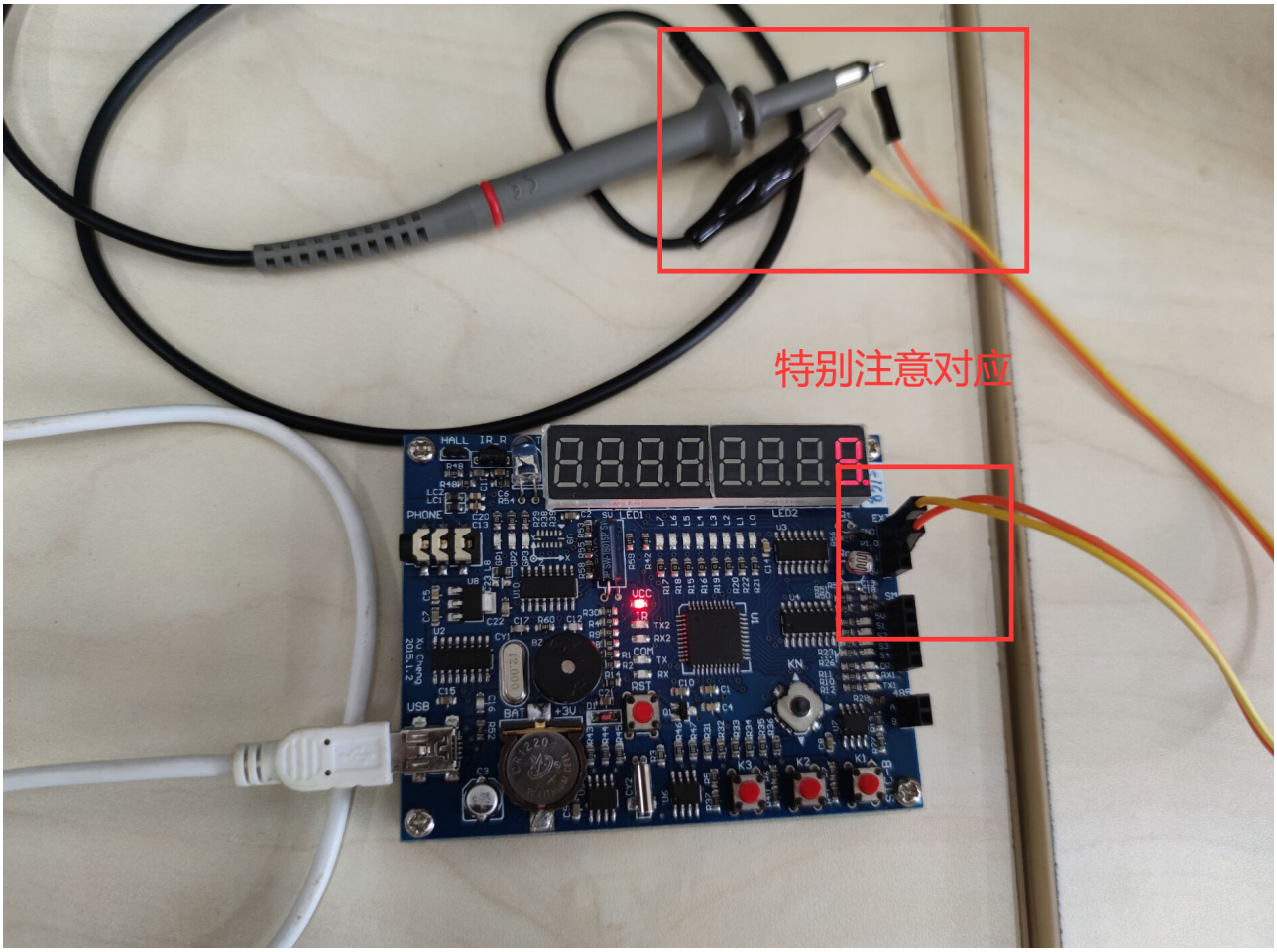


（2）连接

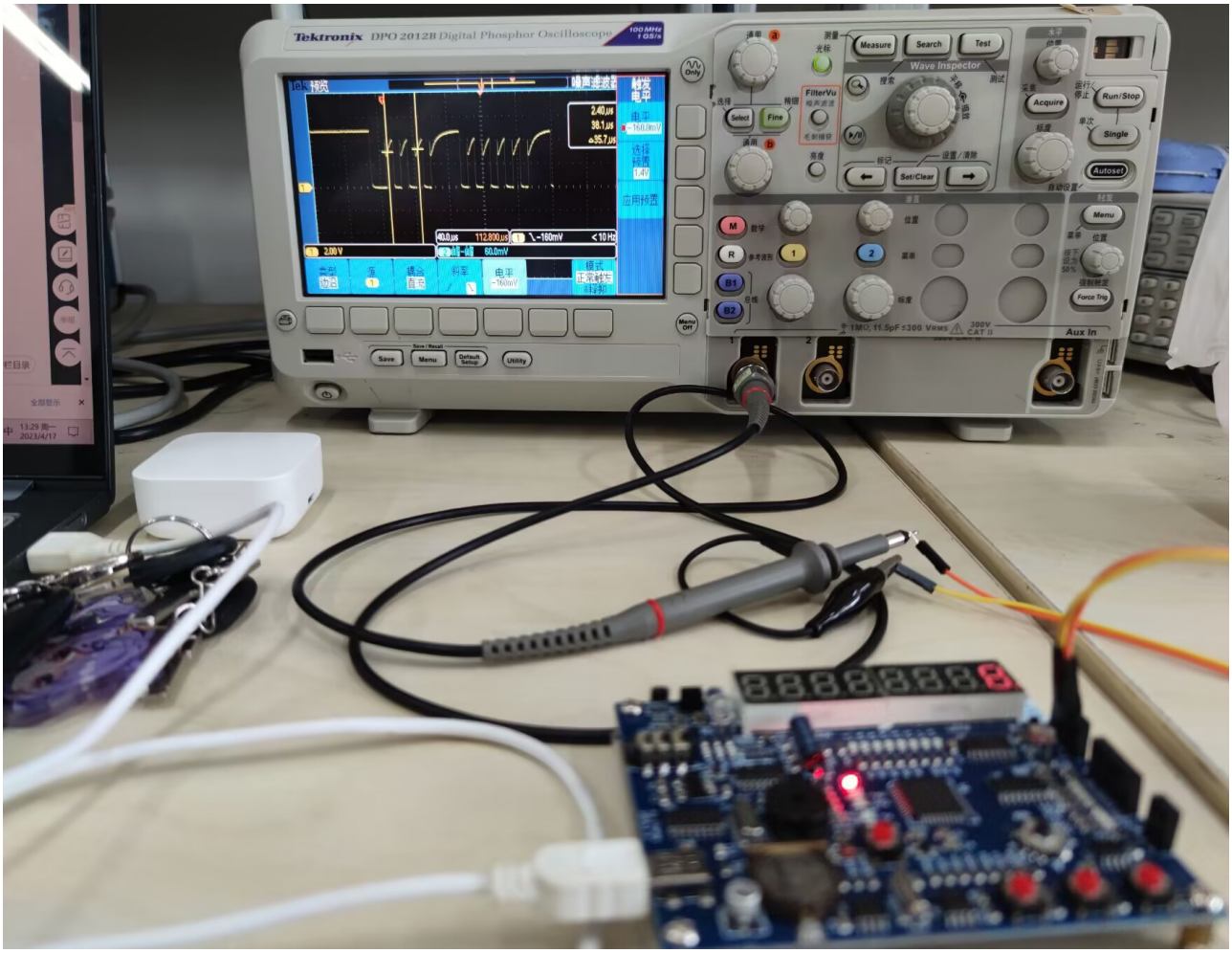
按照指导书步骤，将STC单片机与示波器相连如下（注意接口）

【请特别注意探针的倍率！！！一定要置于×1而不是×10！！！】

【血的教训，捣腾了半节课才发现】

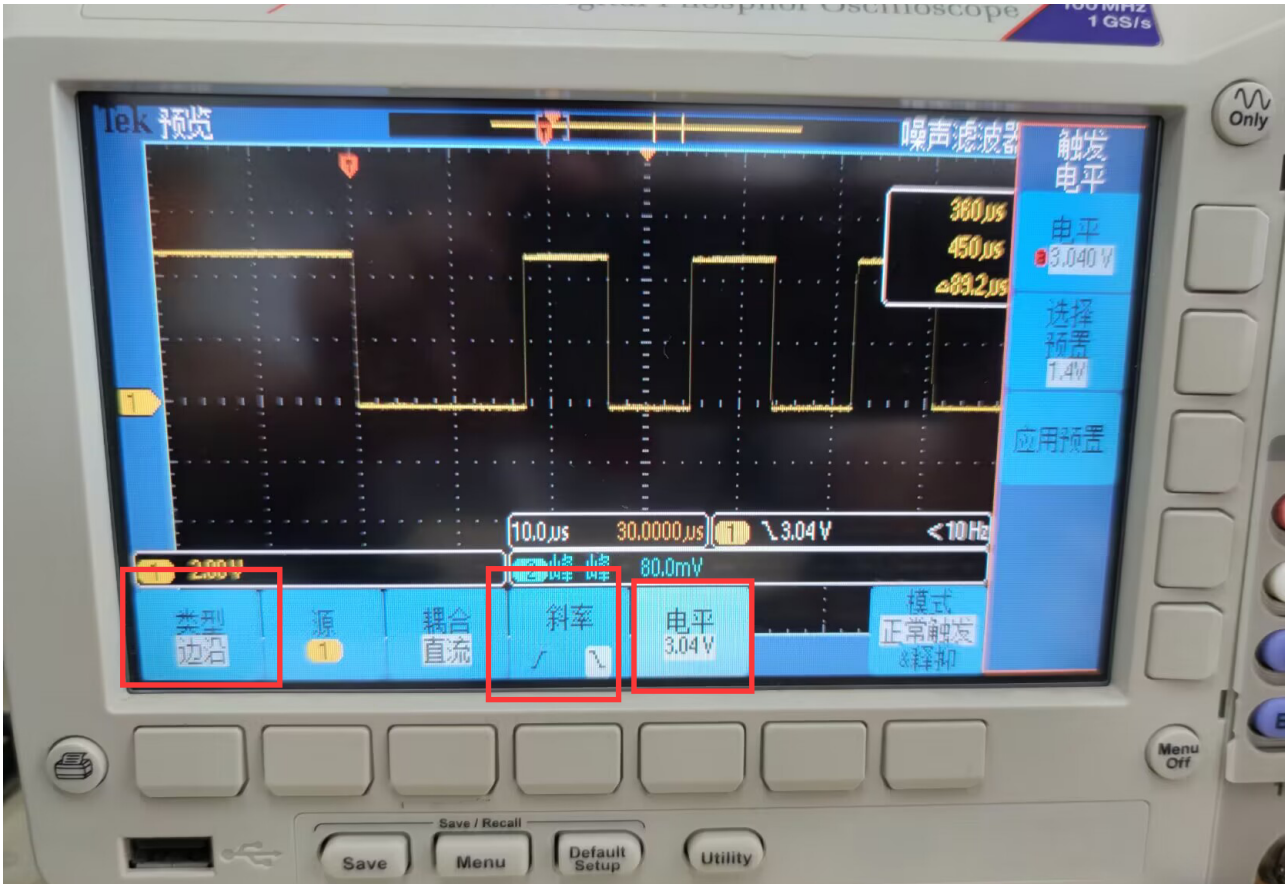


这是连接之后的整体照片。



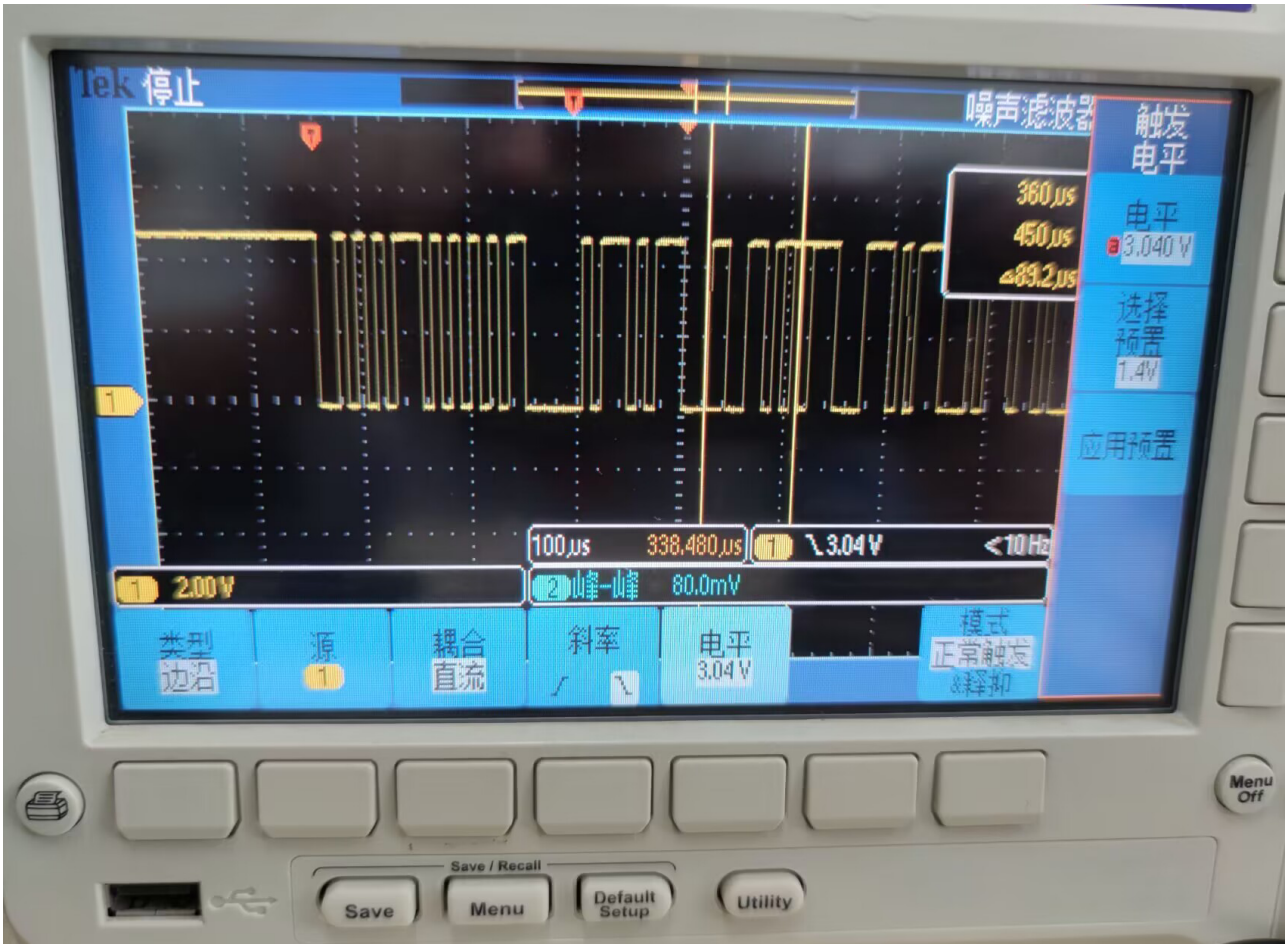
（3）预设值示波器

按照指导书要求，对示波器进行预先设置如下。



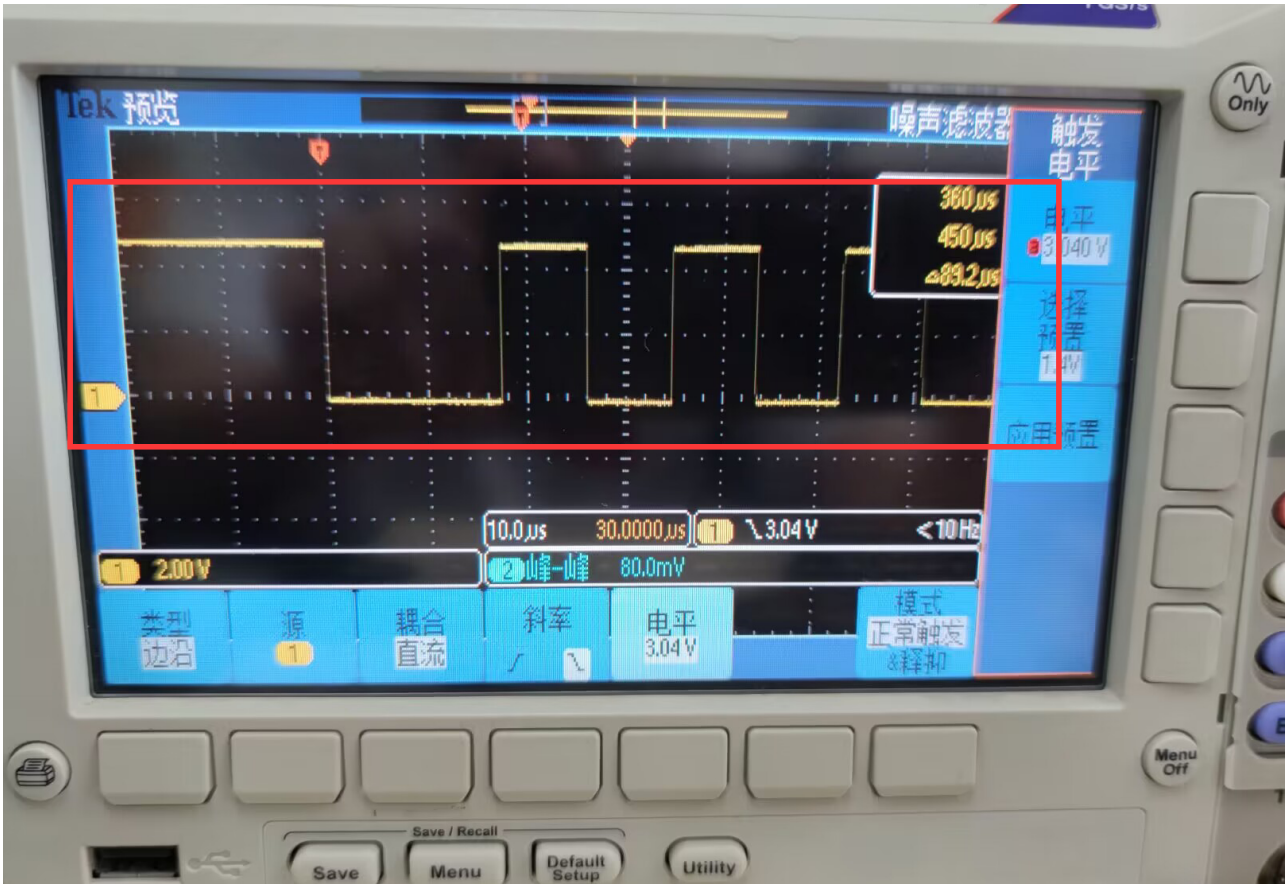
（4）显示单次示结果

待大致能看到一次的发送数据条纹后，按下“single”键，显示一次示波结果。



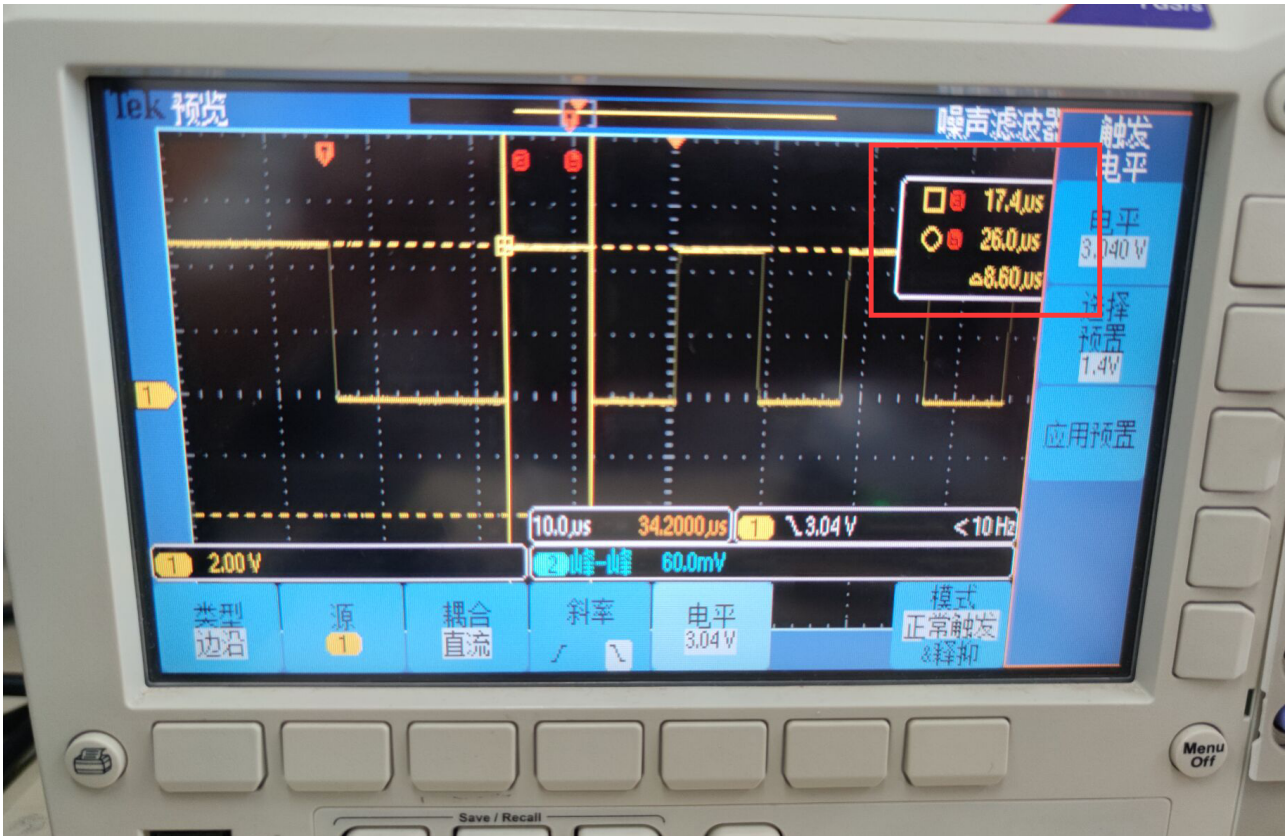
（5）放大示波结果

放大，直到一次波形大约能占到一格，如下。



（6）调节光标并读出差值

调节光标，较为准确地测出下降沿与上升沿的相差



（7）计算波特率

根据波特率的定义，即1s内发送的位数，计算波特率。

1÷8.6μs=116279，符合指导书中提供参考数据的115200。可以看出其实很接近。

【这里千万要注意，我们的测量存在误差，要取接近的，不然后面读取会失败】

【指导书中指出：常用的波特率有1200、1800, 2400, 4800, 9600,19200, 38400, 57600, 115200等。*这次实验*所用的波特率都是*常用波特率*。】

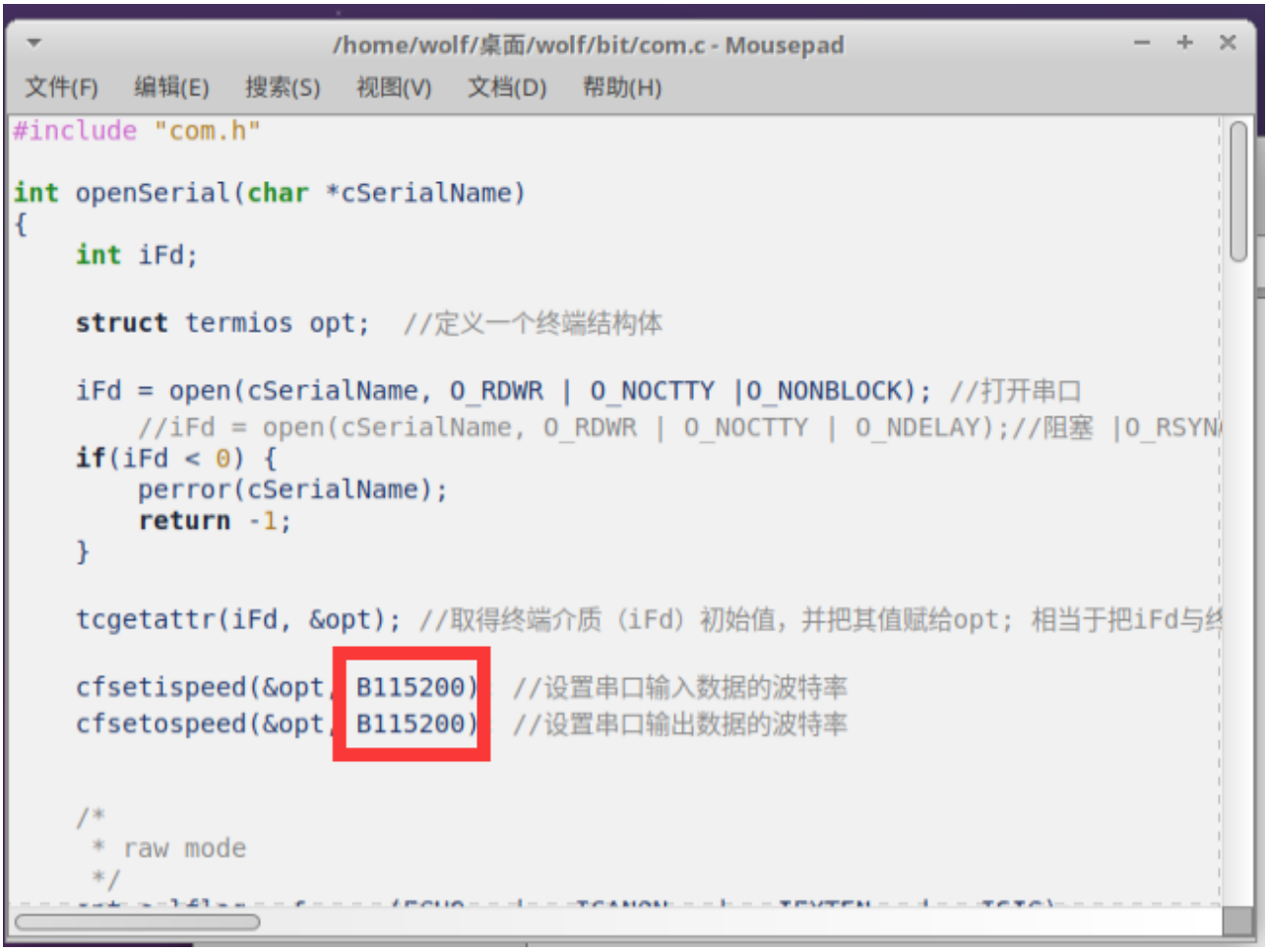
【另外：不同同学得出的波特率不一定一样，所以没必要自己觉得自己错了，只要操作规范，结果合理，符合常用波特率即可】

（9）编写c代码

根据单片机串口波特率，编写C语言程序从虚拟机的串口读取信息。

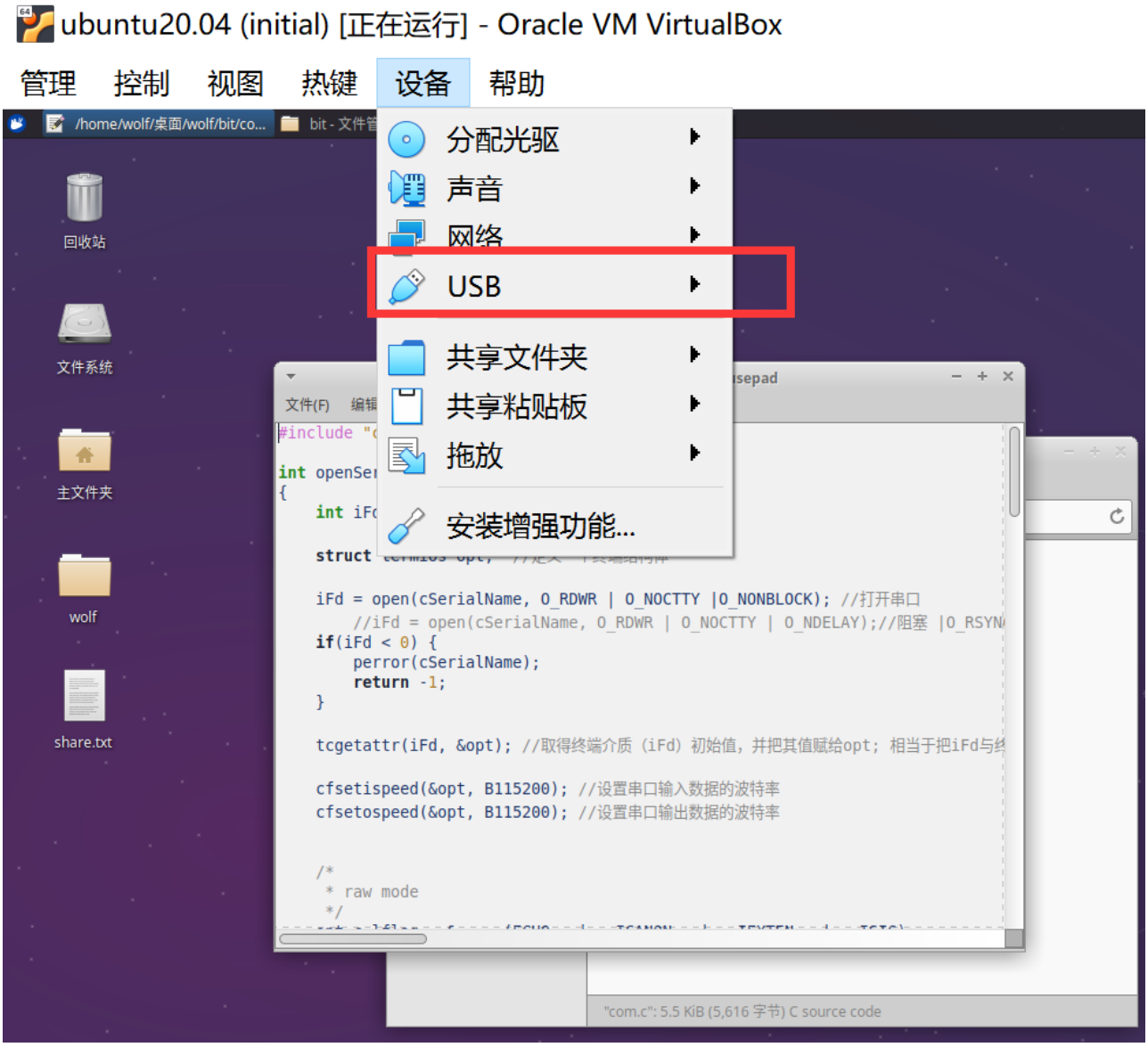
这里我直接套用了指导书中提供的文件，但注意改动波特率为自己测得的波特率相近的参考值，否则无法读取。

只需要改动com.c文件的这个位置即可，而且必须是之前给出参考波特率中的一个。



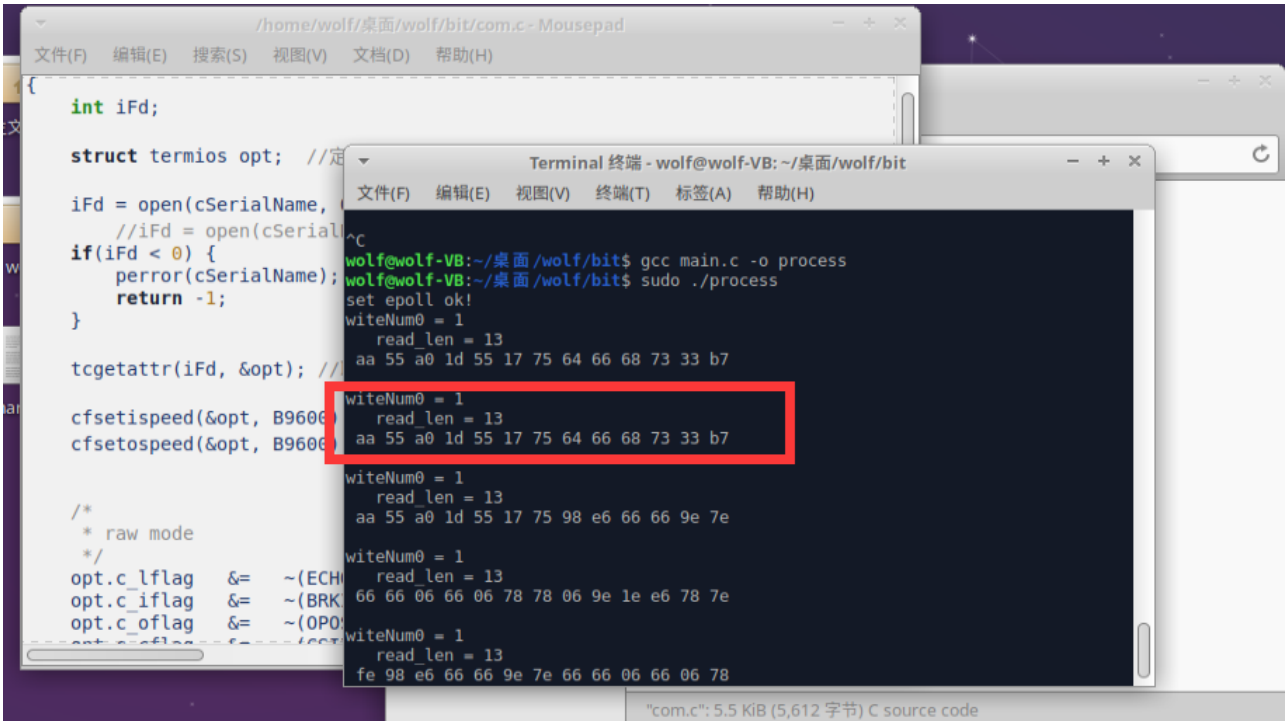
（10）设置虚拟机

设置，使虚拟机可以连接板子。



（11）使用curl提交结果

改动之后，读取信号并提交服务器



如上，下面解释读取到的信息的含义

根据指导书：

【STC单片机计算串口发出来的内容格式为：

0xAA 0x55 序列号

以上内容为二进制数据表示，共13个字节。例如：

0xAA 0x55 0x11 0x22 0x33 0x44 0x55 0x66 0x77 0x88 0x99 0xAA 0xBB 。

其中0x11 0x22 0x33 0x44 0x55 0x66 0x77 0x88 0x99 0xAA 0xBB 就是该单片机的序列号。请用程序读出所领单片机的序列号。】

可知，这里aa 55是开始的序列号，而后面的 a0 1d 56 5c 74 71 b6 69 2d e0 fc即为传输的11位数据。

这里要注意，提交答案的时候不要把aa,55放入，否则是错误的。

提交成功显示“ok”，失败则为“error”。

（12）python任务

(选做任务)熟悉Python语言的同学，也可以使用pyserial模块操作串口读取序列号。下面是不完整的示例代码。

```
import serial #导入pyserial模块

ser=serial.Serial("/dev/ttyUSB0",9600,timeout=0.5)  #导入pyserial模块

reading = self.serial.read_all() #读取串口数据

for i in range(0,len(reading)):  #循环处理串口收到的数据

   data = (int(binascii.hexlify(reading[i]),16))

   print(data)
```

这一部分暂不完成，在下一个实验中会有完整版的。

3.Linux下的设备文件操作

一、实验目的

1、熟练使用Linux下io函数read、write和epoll等

2、熟练处理流式通信数据

二、实验过程

（1）向STC单片机下载下面的程序

下载程序后，单片机将使用1200波特率发送自身序列号，格式同上一节课（4.3节内容）相同。请记录下该序列号。

在一个循环节内，可记录机器序列号为

```
AA 55 A0 1D 55 17 75 64 66 4D 6B 42 9C
```

（机器序列号为AA55后面的部分）

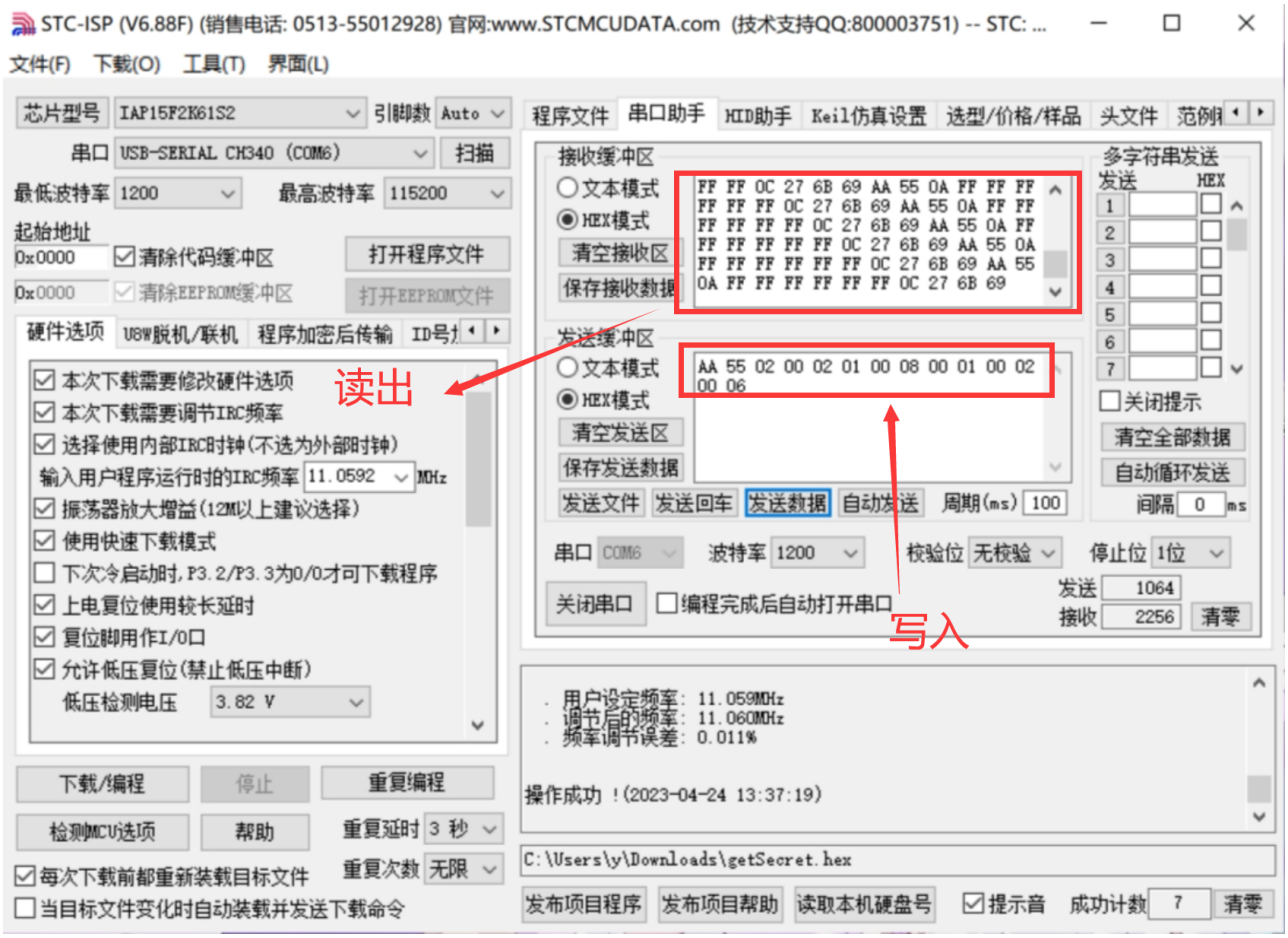
（2）向串口写入自己的学号

向串口写入自己的学号，格式为0xAA 0x55 十二位学号数字。

例如学号20220110203，应该通过串口发送以下数据

AA 55 02 00 02 02 00 01 01 01 00 02 00 03

这里我们使用这个工具自带的串口助手来完成这个步骤



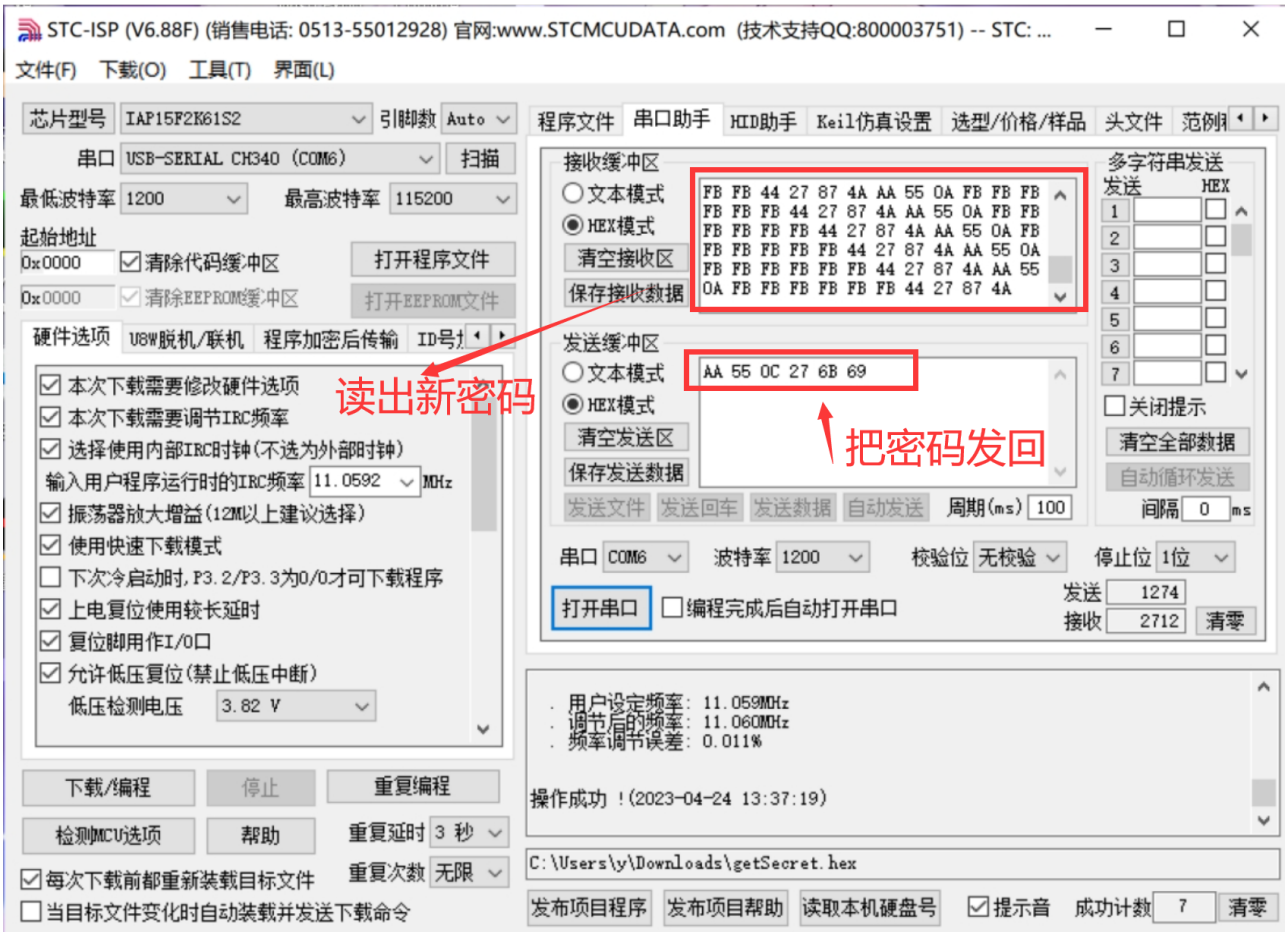
可以读出第一个密码序列为

```
AA 55 0A FF FF FF FF FF FF 0C 27 6B 69
```

（3）解析密码并发回串口

STC单片机接收到学号后会发送第一串密码，密码长度为4字节。请将解析出该串密码并原样发回给串口。STC单品机收到返回后会继续发送下一串密码，请继续解析出该串密码并原样发回给串口。以此往复，将收到的最后一串密码记录下来。

> 例如，收到以下密码串
>
> *AA 55 0A 00 00 00 00 00 00 C9 34 3F 5D*
>
> 其中*AA 55*为前导串，*0A*表示起始字节是第*10*字节。请将密码原样发回给串口。
>
> *AA 55 C9 34 3F 5D*

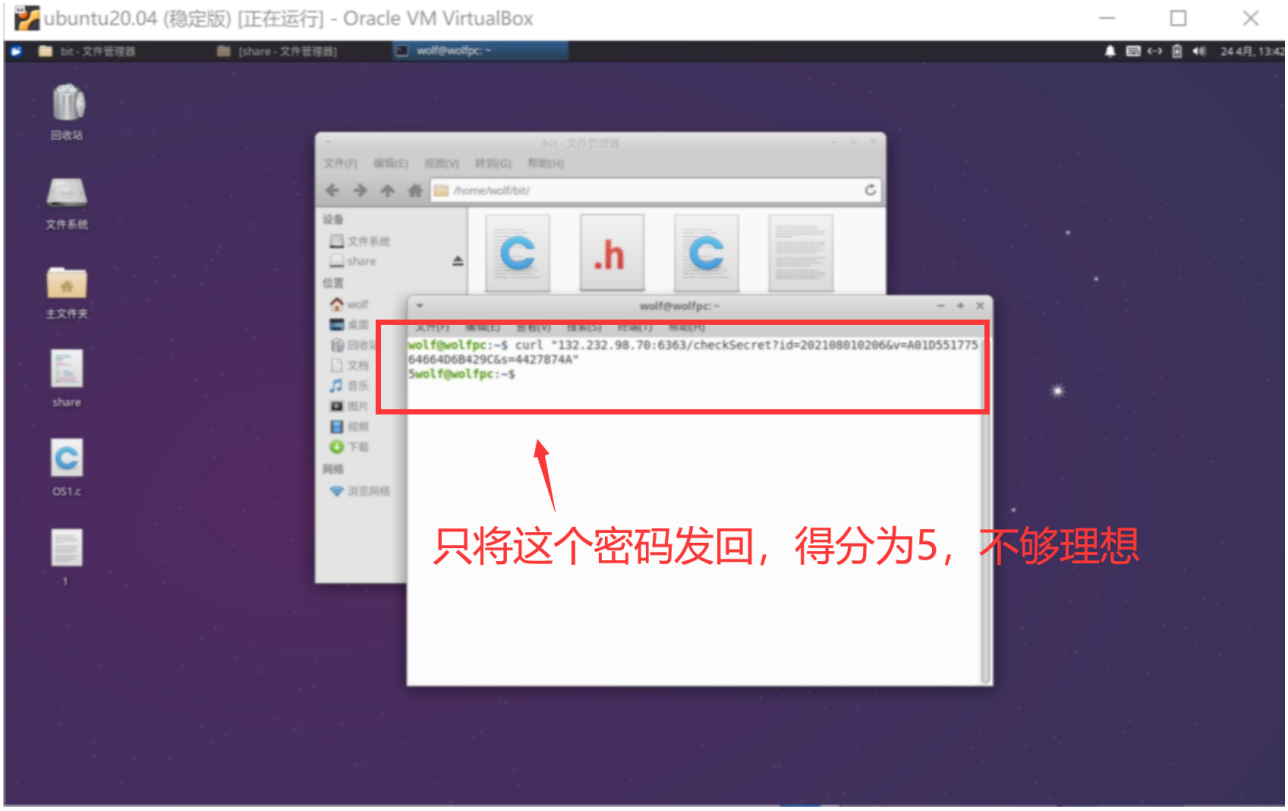


可读出新密码为

```
AA 55 0A FB FB FB FB FB FB 44 27 87 4A
```

（4）使用curl提交结果

> 将学号、序列号、最后一串密码发送到课程后台。上送命令语法为
>
> *curl "132.232.98.70:6363/checkSecret?id=学号&v=序列号&s=密码"*
>
> 例如，序列号是*A01D564D744A42C9363F5E*，学号是*202201110203*，最后一串密码为*51ED8D3A*，那么上报结果命令为：
>
> *curl "132.232.98.70:6363/checkSecret?id=202201110203&v=A01D564D744A42C9363F5E&s=51ED8D3A"*



（5）(选做)python编程完成

在Linux系统下使用C++或者Python代码完成上面第三步动作，并将取到的最后一组密码发送到课程后台中。

这里出了点问题，我的ubuntu没有装pyserial，所以我先安装了一个pip，然后再安装serial，若出现“网络不可达”，则注意使用镜像源安装，我们使用的是华科大的镜像源。

```
wolf@wolfpc:~/桌面$ sudo pip3 install pyserial -i https://pypi.mirrors.ustc.edu.
cn/simple/
Looking in indexes: https://pypi.mirrors.ustc.edu.cn/simple/
Collecting pyserial
  Downloading https://mirrors.bfsu.edu.cn/pypi/web/packages/07/bc/587a445451b253
b285629263eb51c2d8e9bcea4fc97826266d186f96f558/pyserial-3.5-py2.py3-none-any.whl
 (90 kB)
     |                                | 90 kB 1.0 MB/s
Installing collected packages: pyserial
Successfully installed pyserial-3.5
```

程序如下：

```
import serial.tools.list_ports
# 字符串转字符串
def b2s(data):
    return ''.join([f'{ch:0>2x}' for ch in data]).upper()
# 连接设备
ser = serial.Serial(list(serial.tools.list_ports.comports()[0])[0],
1200)
```

```
# 判断是否连接到设备
assert (ser != None)
# 输入学号
studentID = input("学号:")
# 判断学号是否正确
assert (studentID.isdigit() and len(studentID) == 12)
# 读入序列号
number = b2s(ser.read(13))
# 断言魔数为AA55
assert (number[:4] == "AA55")
# 取出序列号
number = number[4:]
assert (len(number) == 11 * 2)
print(f'序列号:{number}')
# 使用学号构造即将发送的字节数据
startData = b'\xaa\x55' + bytes([ord(ch) - ord('0') for ch in
studentID])
# 写入设备
ser.write(startData)
password = b''  # 读到的密码
i = 1  # 循环次数
# 开启永真循环，直到ctrl+c被按下
while True:
    # 读1位，探测魔数
    t = b2s(ser.read(1))
    # 第一个魔数应该是AA
    if t != "AA":
        continue
    # 再读一位
    t = b2s(ser.read(1))
    # 第二个魔数应该是55
    if t != "55":
        continue
    # 判断魔数后可以放心读取数据
    # 读入密码开始位置的索引
    index = list(ser.read(1))[0]
    # 若索引大于4，则表明中间有无关字节，吸收掉无关字节
    if index - 4 > 0:
        ser.read(index - 4)
    # 读入四字节的密码
    password = ser.read(4)
    print(f'[{i}]密码:{b2s(password)}')
```

```
    # 使用这次读到的密码构造数据，发送给设备以读取下一次的密码
    ser.write(b'\xaa\x55' + password)
    i += 1
```

一定要在sudo权限下运行main.py，输入自己的学号，得到结果

```
wolf@wolfpc:~/桌面$ sudo python3 main.py
学号:202108010206
序列号:A01D55177564664D6B429C
[1]密码:1D551775
[2]密码:FFFFFFFF
[3]密码:0C276B69
[4]密码:09315562
[5]密码:1E75C8EB
[6]密码:BC5C09F3
[7]密码:4427874A
[8]密码:DCC22B1C
[9]密码:F47E9BA5
[10]密码:6B4A6091
[11]密码:B26FCD48
```

（可以发现我们上面读出的两段密码也在这个序列中，只是分值较小）

程序会逐行读取每一个密码，达到256时密码不再变化，此时的密码就是最终的答案。

```
[245]密码:B51C65A0
[246]密码:DF369C9D
[247]密码:E5270771
[248]密码:989B2A8B
[249]密码:8B35A4AE
[250]密码:404C6EDA
[251]密码:143CC404
[252]密码:3048A6C1
[253]密码:CDEBF93B
[254]密码:7E79F16F
[255]密码:6ABFD7E8
[256]密码:41E7D621
[257]密码:8EE67359
[258]密码:9CCFCC42
[259]密码:4B6B409C
[260]密码:4B6B409C
[261]密码:4B6B409C
[262]密码:4B6B409C
[263]密码:4B6B409C
[264]密码:4B6B409C
[265]密码:4B6B409C
[266]密码:4B6B409C
[267]密码:4B6B409C
[268]密码:4B6B409C
```

使用curl "132.232.98.70:6363/checkSecret?id=202108010206&v=A01D55177564664D6B429C&s=4B6B409C"提交即可。

```
wolf@wolfpc:~/桌面$ curl "132.232.98.70:6363/checkSecret?id=202108010206&v=A01D5
5177564664D6B429C&s=4B6B409C"
256wolf@wolfpc:~/桌面$
```

返回256，表示已经拿到最高分，实验结束。

4.RS485信号的测量

【特别注意序列号随着下载程序改变而改变，不可再用之前的方法得出序列号或者沿用之前的序列号！】

（1）为A板下载新的hex文件。

然后按下K3按键，电路板将通过RS485向外发出一组序列号信息。该信号序列格式为：

0xAA 0x55 4字节序列号

【这里可以直接通过示波器读出，但是有点难看，我选择的是让其返回至电脑直接读出。】在后面完成。

请在板上RS485接口插上杜邦线，然后示波器观察该序列信息的内容和波特率并做好记录。

下图是A板的示波器显示，使用1÷424μs，其波特率为2400

（2）下载B板文件

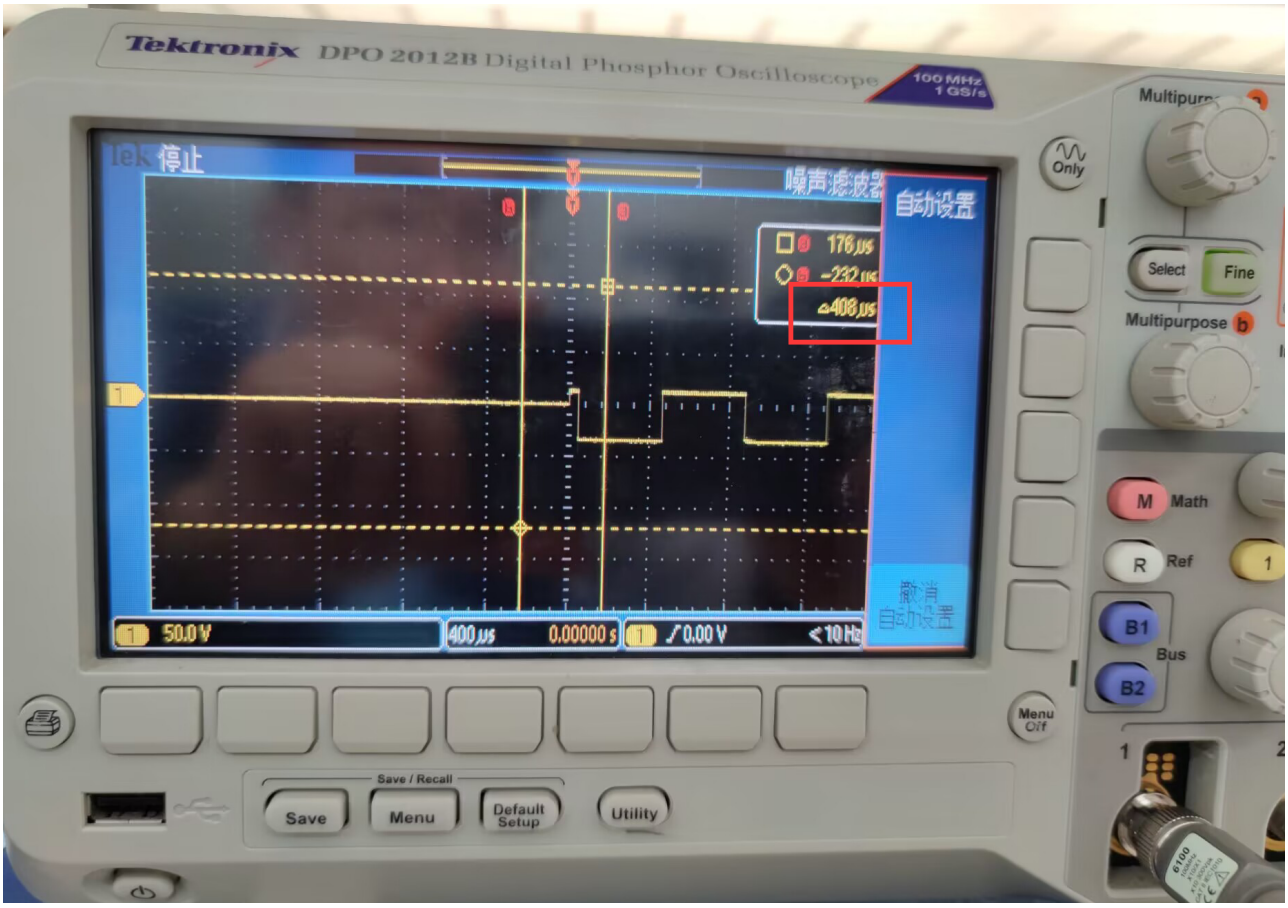
使用同学的STC电路板(后面称为B板)，下载B板Hex文件：

B板会将计算机通过USB 串口发送的数据，转发到RS485接口上。同时将RS485接口收到的数据，通过USB 串口转发到计算机上。我们接下来使用这个B板与自己的A板进行通信，读取A板的密码。

B板每次上下拨动摇杆，其串口和RS485总线上波特率会发生变化，同时会在RS485接口上输出一个0x55。请插上杜邦线后，使用示波器观察其波特率。

请通过上下拨动B板摇杆调节B板波特率，使之与A板输出的RS485波特率相匹配。

这里我们上下拨动B板摇杆，直至差不多能等于上面的424μs为止，完成时结果如下。

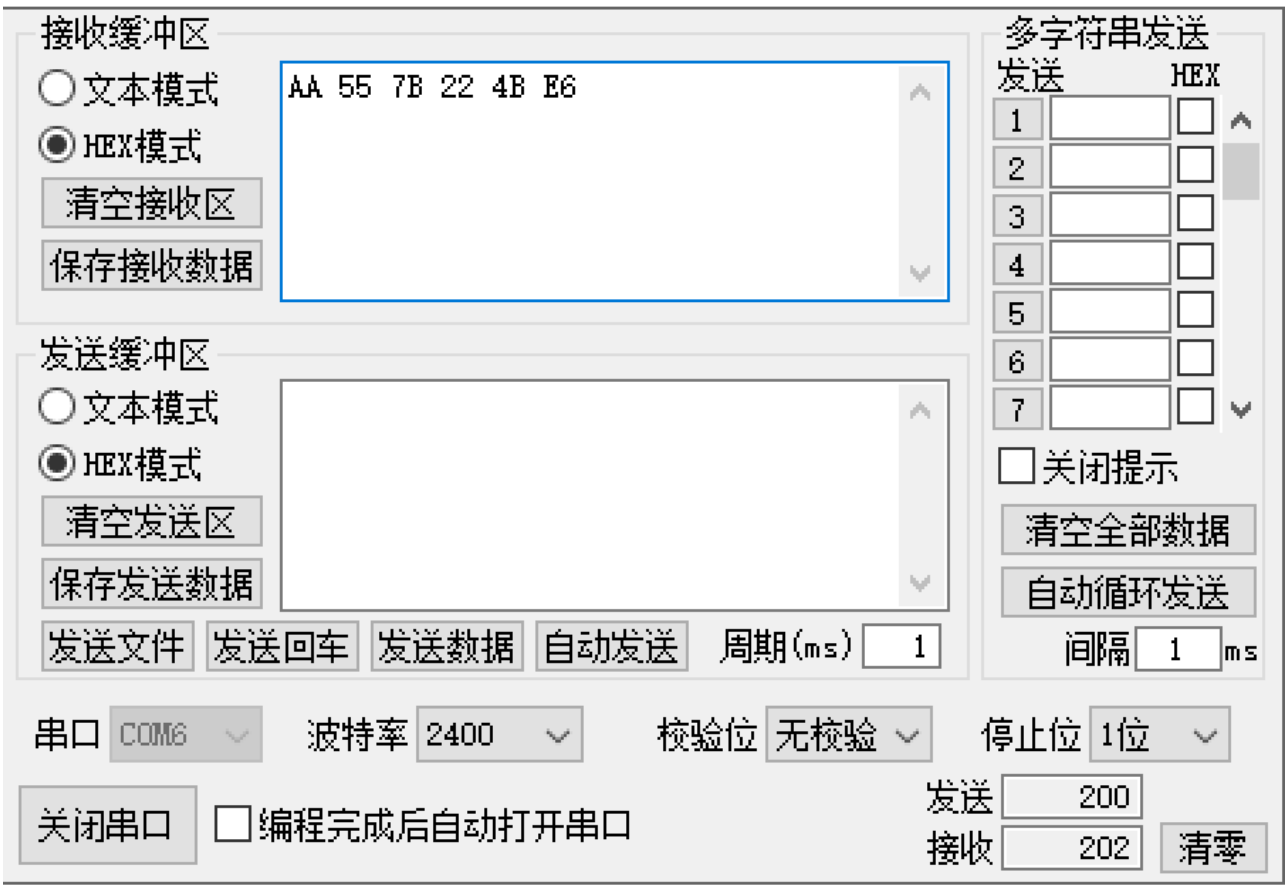


（3）补充完成获取A板序列号

之前为了更加清晰，选择在此完成读取序列号

在A板、B板的波特率相同后，按下A板K3键，返回序列号（请一定注意是4字节）如下：

> *7B 22 4B E6*



（4）向串口写入读密码指令

调用write函数接口向串口发送读密码命令。

读密码命令格式为：0xAA 0x55 A板序列号 12字节学号

我发送的读取密码命令如下

> *AA 55 7B 22 4B E6 02 00 02 01 00 08 00 01 00 02 00 06*

注意，发送时所使用的波特率应该与A板通信波特率一致。命令通过B板转换为RS485信号发送到A板。

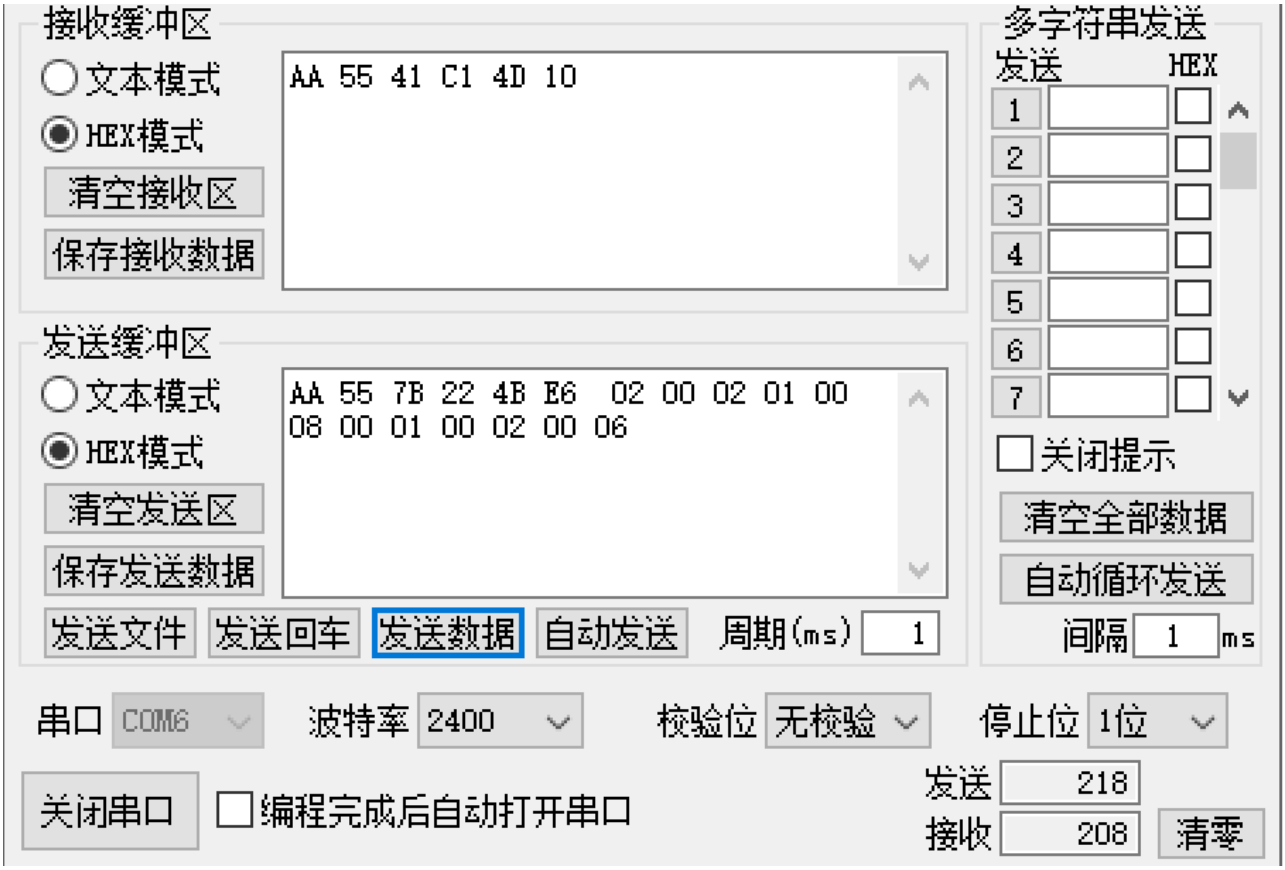
A板会回应密码，回应格式为：

0xAA 0x55 4字节密码

回应密码会经过B板转发到计算机上。请调用read函数接口从串口读取该密码。

返回答案（请一定注意是6字节）：

> *91 99 35 AD 09 71*



（5）使用curl提交结果

将A板序列号、学号和密码通过curl命令发送到课程后台系统。

上送命令语法为

> *curl "132.232.98.70:6363/check485?id=学号&v=序列号&s=密码"*

回应OK表示正确，其它表示错误。

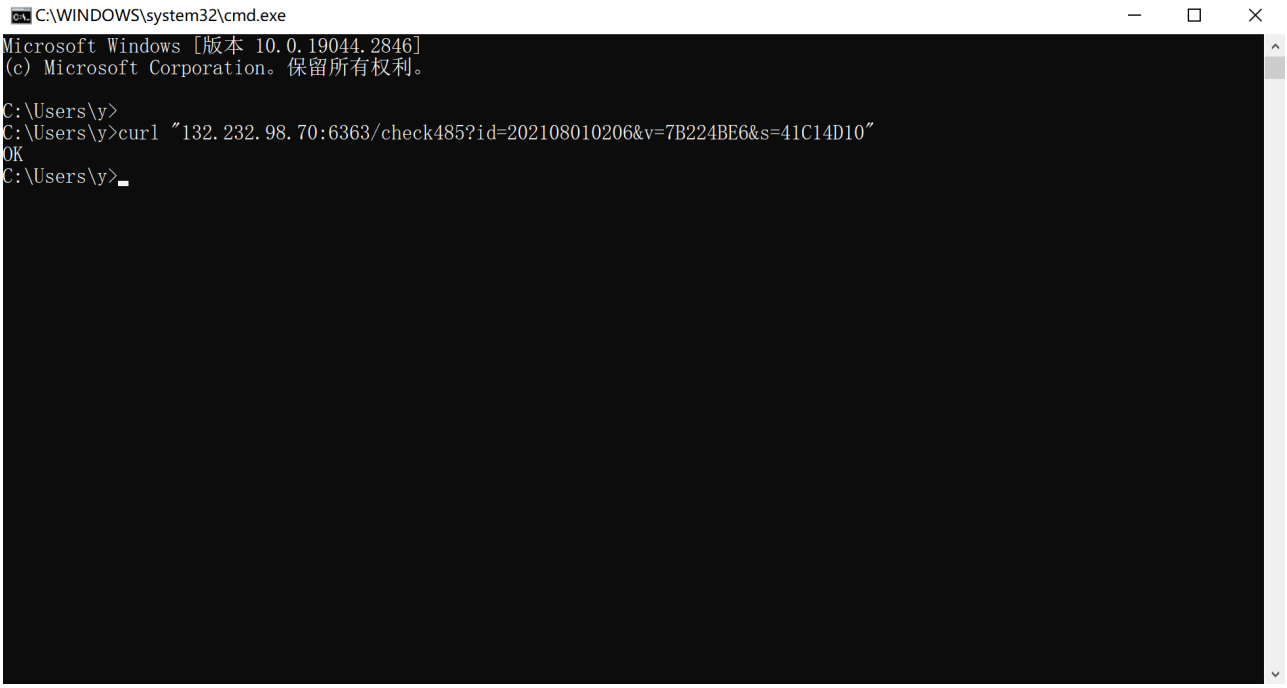
> 例如，序列号是*D9 01 49 8A*，学号是*202201030303*，回应的密码为 *01 8A D9 49*，那么上报结果命令为：
>
> *curl "132.232.98.70:6363/check485?id=202201030303&v=D901498A&s=018AD949"*

其中NOK表示失败，DUP表示这个板子已经被其他同学使用过了，需要更换板子继续。

我发送的提交命令如下

> *curl "132.232.98.70:6363/check485?id=202108010206&v=7B224BE6&s=41C14D10"*

返回结果为OK，表示成功



5.RS485总线数据收发

（1）向A单片机板下载下面的程序

> *AHasSecrets.hex*

向B单片机板下载下面的程序

> *B_Relay.hex*

下载程序后，A板单片机将使用1200波特率约每300ms发送一次自身序列号，格式为0xAA 0x55 4字节序列号

我获取的序列号是

> *CE 19 8F 4C*

（2）向B板写入学号

通过B板串口向RS485总线写入自己的学号，格式为0xAA 0x55 十二位学号数字。

例如学号20220110203，应该通过串口发送以下数据AA 55 02 00 02 02 00 01 01 01 00 02 00 03

我需要发送的是

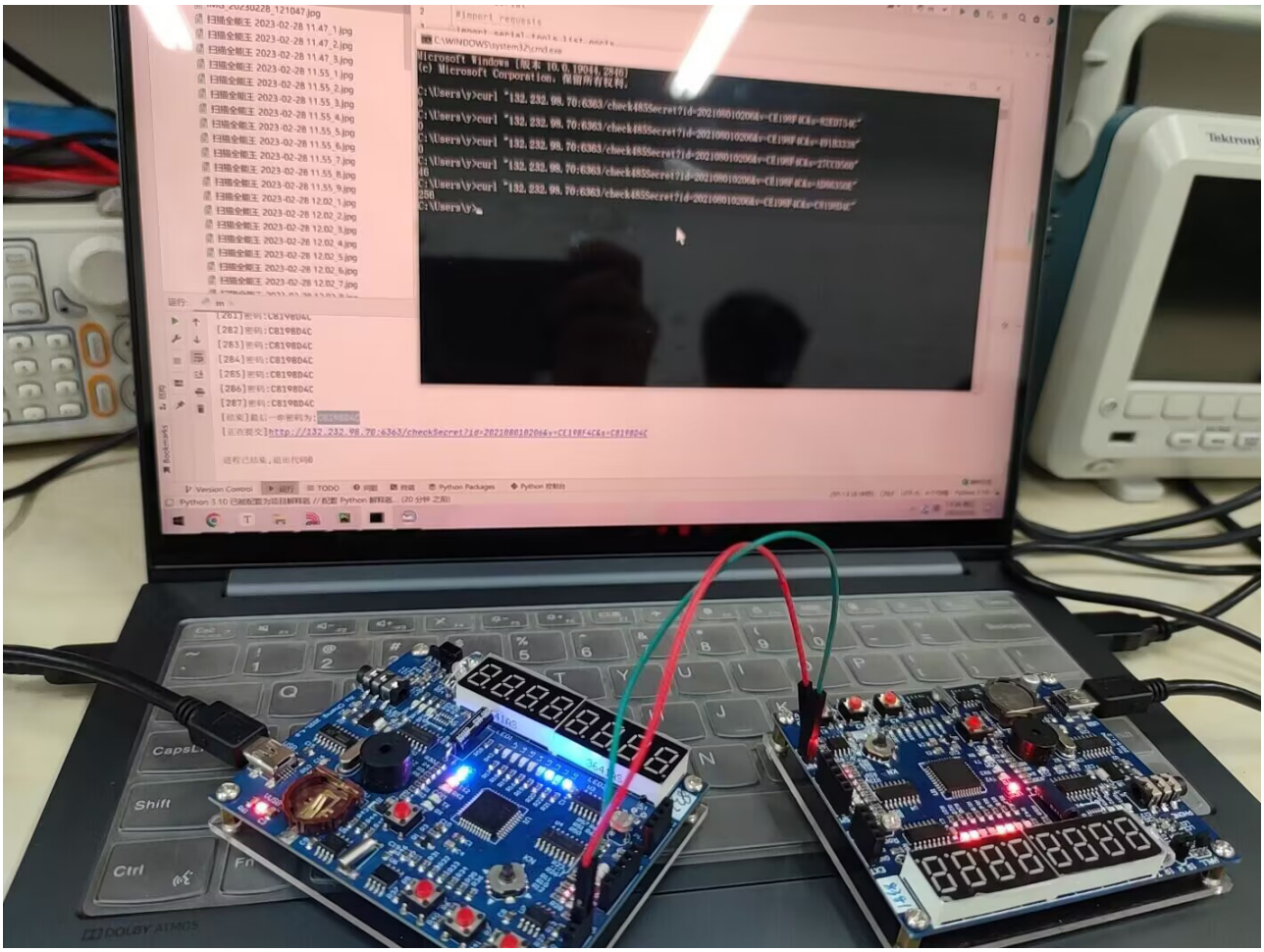
> *AA 55 02 00 02 01 00 08 00 01 00 02 00 06*

（3）使用A板发送密码

A板单片机接收到学号后会每300ms发送第一串密码，密码长度为4字节。请将解析出该串密码并在150ms内原样发回给串口（否则会造成总线的冲突）。STC单品机收到返回后会继续发送下一串密码，请继续解析出该串密码并在150ms内原样发回给串口。以此往复，将收到的最后一串密码记录下来。

例如，收到以下密码串AA 55 00 00 00 00 00 C9 34 3F 5D

其中AA 55为前导串，最后4个字节是密码。请将密码原样发回给串口。即发送AA 55 C9 34 3F 5D



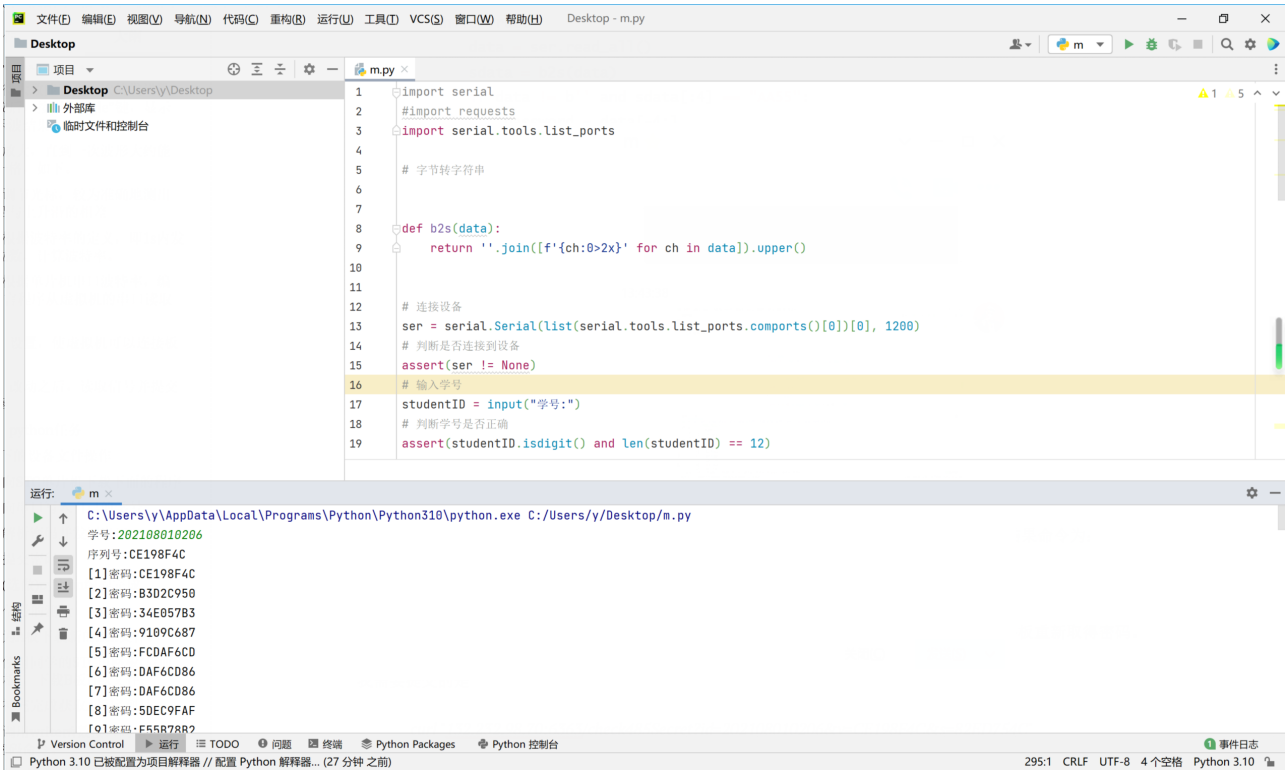
（4）编写python代码并获取结果

使用python编程解决，程序如下

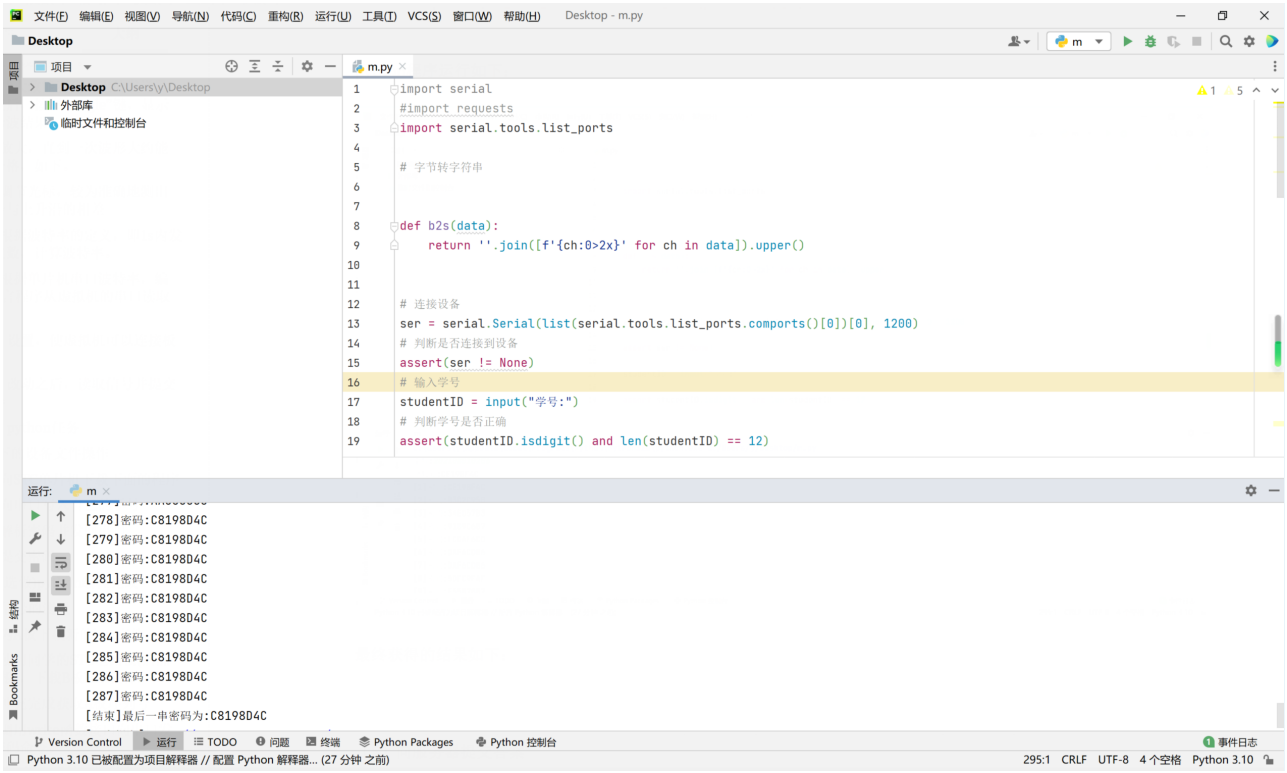
```
import serial
import serial.tools.list_ports
# 字节转字符串
def b2s(data):
    return ''.join([f'{ch:0>2x}' for ch in data]).upper()
# 连接设备
ser = serial.Serial(list(serial.tools.list_ports.comports()[0])[0],
1200)
# 判断是否连接到设备
assert(ser != None)
# 输入学号
studentID = input("学号:")
# 判断学号是否正确
assert(studentID.isdigit() and len(studentID) == 12)
# 读入序列号
number = b2s(ser.read(6))
# 断言魔数为AA55
assert(number[:4] == "AA55")
# 取出序列号
number = number[4:]
# 判断序列号长度是否正确
assert(len(number) == 4 * 2)
print(f'序列号:{number}')
# 使用学号构造即将发送的字节数据
startData = b'\xaa\x55' + bytes([ord(ch) - ord('0') for ch in
studentID])
# 写入数据
ser.write(startData)
password = b''  # 读到的密码
i = 1   # 记录循环次数
# 开启永真循环，直到Ctrl+C被按下
try:
    while True:
        data = ser.read_all()
        sdata = b2s(data)
        if data != b'' and sdata[:4] == "AA55":
            password = data[-4:]
            if len(password) == 4:
```

```
                print(f'[{i}]密码:{b2s(password)}')
                # 使用这次读到的密码构造数据，发送给设备以读取下一次的密码
                ser.write(b'\xaa\x55' + password)
                i += 1
    # 当Ctr+C被按下时执行，建议在读到的密码不再变化时按下，大概在256次循环之后
    except KeyboardInterrupt:
        print(f'[结束]最后一串密码为:{b2s(password)}')
```

python程序运行如下：



最终获得的结果如下：



（5）使用curl提交结果

将学号、序列号、最后一串密码发送到课程后台。上述命令语法为

curl "132.232.98.70:6363/check485Secret?id=学号&v=序列号&s=密码"

例如，序列号是2C9363F5E，学号是202201110203，最后一串密码为51ED8D3A，那么上报结果命令为：

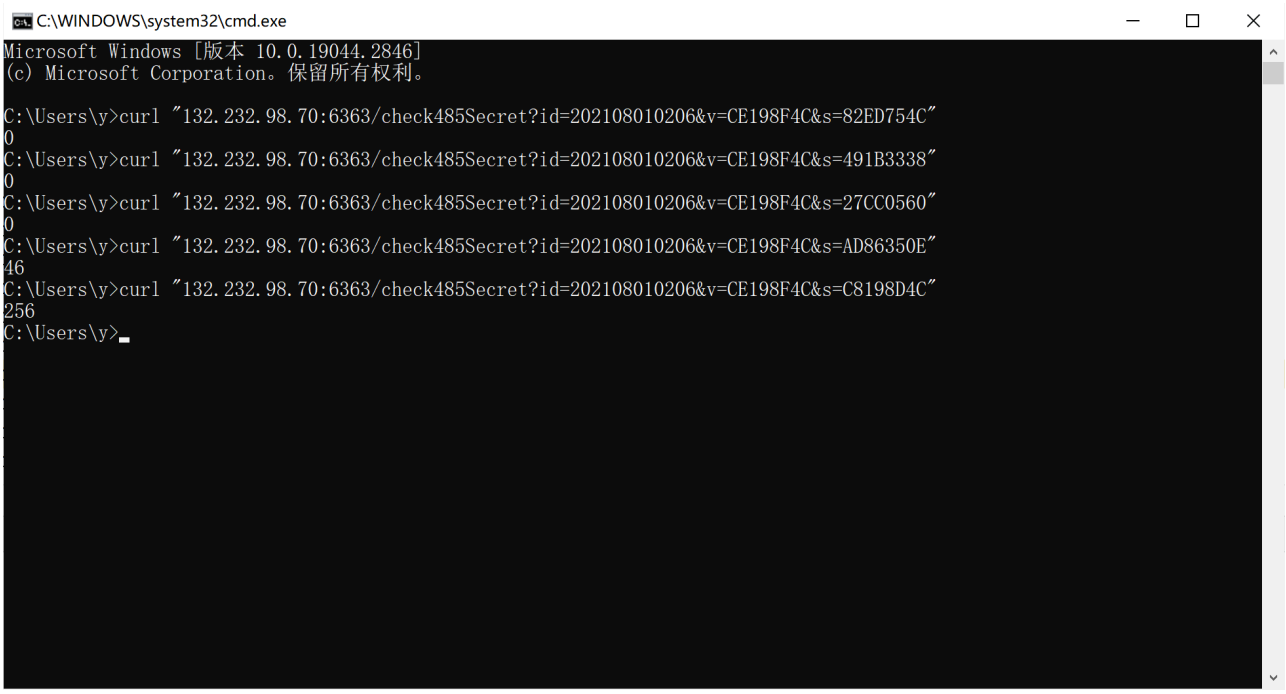
curl "132.232.98.70:6363/check485Secret?
id=202201110203&v=2C9363F5E&s=51ED8D3A"

后台返回的数字表示该密码的编号，返回DUP表示该序列号已经被其它同学使用，请更换电路板重新取得密码。

我需要提交的是

> *curl "132.232.98.70:6363/check485Secret?*
> *id=202108010206&v=CE198F4C&s=82ED754C"*

使用curl指令提交答案，获取分数256。



实验总结

通过这次实验，我能够层层深入地了解串口相关的知识并能够进行实操。

我经常与小伙伴一起从中午的十二点半一直做到下午的两三点。花费在串口实验上的时间超过了8个小时。

在实验中，我进一步熟悉了linux操作系统的使用。熟悉了单片机下载程序，调试串口的方法，理解了使用IO函数实现的串口数据的收发。编写串口数据收发程序也进一步体会了如何对串口数据进行处理。另外地，我通过实验进一步熟悉了串口通信的方法，以及使用write，read等函数进行串口数据进行读写。我了解了RS485接口的使用，通过RS485实现了两板的通信和数据传输。认识了RS485的半双工通信模式，在程序运行时，有些产生的密码是错误的，即发生了冲突得到了错误数据。对于程序的编写，能够通过python语言程序实现功能，后续还需要加深理解，提高代码的性能以及编写相关代码的能力。最后说一句，python是真的管用。

串口相关的知识是我感觉在电子测试平台与工具中是最有趣的知识。因为我不仅能够学到怎么让两个供电的小板子进行相互的通信，也能够输出结果与写入需求。最为重要的是，我能够在一次学习到知识后立刻通过实验来验证知识的掌握程度，通过curl提交来获取自己的分数，这都是极好的。

未来，我会继续努力学习串口相关的学习知识，并付诸实践。